```
Client.order(created_at: :asc)
# OR
Client.order("created_at DESC")
# OR
Client.order("created_at ASC")
```

Or ordering by multiple fields:

```
Client.order(orders_count: :asc, created_at: :desc)
# OR
Client.order(:orders_count, created_at: :desc)
# OR
Client.order("orders_count ASC, created_at DESC")
# OR
Client.order("orders_count ASC", "created_at DESC")
```

If you want to call `order` multiple times e.g. in different context, new order will append previous one

```
Client.order("orders_count ASC").order("created_at DESC")
# SELECT * FROM clients ORDER BY orders_count ASC, created_at DESC
```

4 Selecting Specific Fields

By default, `Model.find` selects all the fields from the result set using `select *`.

To select only a subset of fields from the result set, you can specify the subset via the `select`method.

For example, to select only `viewable_by` and `locked` columns:

```
Client.select("viewable_by, locked")
```

The SQL query used by this find call will be somewhat like:

```
SELECT viewable_by, locked FROM clients
```

Be careful because this also means you're initializing a model object with only the fields that you've selected. If you attempt to access a field that is not in the initialized record you'll receive:

```
ActiveModel::MissingAttributeError: missing attribute: <attribute>
```

Where `<attribute>` is the attribute you asked for. The `id` method will not raise the `ActiveRecord::MissingAttributeError`, so just be careful when working with associations because they need the `id` method to function properly.

If you would like to only grab a single record per unique value in a certain field, you can use `distinct`:

```
Client.select(:name).distinct
```

This would generate SQL like:

```
SELECT DISTINCT name FROM clients
```

You can also remove the uniqueness constraint:

```
query = Client.select(:name).distinct
# => Returns unique names

query.distinct(false)
# => Returns all names, even if there are duplicates
```

5 Limit and Offset

To apply `LIMIT` to the SQL fired by the `Model.find`, you can specify the `LIMIT` using `limit` and `offset` methods on the relation.

You can use `limit` to specify the number of records to be retrieved, and use `offset` to specify the number of records to skip before starting to return the records. For example

```
Client.limit(5)
```